

Game Architecture & Best Practices

Questo documento centralizza tutte le best practices, pattern architetturali e linee guida per lo sviluppo del gioco HTML5 singleplayer.

Indice

-  [Architettura Generale](#)
 -  [Frontend Development](#)
 -  [Backend Development](#)
 -  [Security & Validation](#)
 -  [Monitoring & Analytics](#)
 -  [Performance Optimization](#)
 -  [Testing Strategy](#)
 -  [Deployment & CI/CD](#)
 -  [Development Workflow](#)
-

Architettura Generale

Comunicazione tra Servizi

Il sistema utilizza una rete Docker Compose strutturata con isolamento di sicurezza:

Network Segmentation:

- **internal_net**: Comunicazione sicura backend-database (isolata da internet)
- **proxy_net**: NGINX Proxy Manager ↔ Backend services
- **frontend_net**: NGINX Proxy Manager ↔ Frontend applications
- **kibana_net**: Kibana con accesso internet per Elastic Package Registry

Service Communication:

- **Frontend → Server**: HTTP via NGINX Proxy Manager (**localhost** → **server:3000**)
- **Sandbox → Server**: HTTP via NGINX Proxy Manager (mirror del frontend per testing)
- **Frontend → Redis**: Connessione diretta con utente **frontend** (read-only access)
- **Server → MongoDB**: Connessione diretta su **internal_net**
- **Server → Redis**: Connessione con utente **app** (full access)

Redis ACL Configuration

Accesso controllato tramite **docker/redis/users.acl**:

- **user app**: Full access per backend Go server
 - **user frontend**: Read-only access (**+get +exists +ttl +pttl +type +ping**)
 - **user redisui**: Admin access per Redis UI
-

Frontend Development

Stack Tecnologico

Framework: Phaser 3.60.0 per game engine

- TypeScript per type safety
- Vite per build system e hot reload
- Socket.io-client per comunicazione real-time (se necessaria)

Struttura Directory:

- `src/main.ts`: Entry point applicazione
- `src/scenes/`: Game scenes Phaser
- `src/components/`: UI components riutilizzabili
- `src/items/`: Game objects e entities
- `src/network/`: Logica comunicazione server (attualmente vuota)
- `src/assets/`: Risorse statiche (sprite, audio)

Sandbox Environment

Scopo: Mirror del frontend per testing isolato di comportamenti specifici

- **Configurazione:** Identica al frontend ma separata (`sandbox/` directory)
- **Testing Focus:** Scene personalizzate con stats di partita craftate per testing
- **Vite Dev Server:** Porta 5173 per hot reload durante development
- **No Monitoring:** Escluso da APM/RUM tracking per evitare noise nei dati
- **No Persistent Logging:** Log temporanei solo per debugging immediato

Uso Consigliato:

- Prototipazione rapida di nuove game mechanics
- Testing di scenari specifici con dati controllati
- Debugging di comportamenti problematici isolati
- Development di features senza impatto su metrics produzione

Schema Validation Frontend

Gli schemi JSON in `comms/frontend/` definiscono contratti di comunicazione. **Implementazione richiesta:** Validazione tramite Zod per richieste API.

Best Practices:

- Utilizzare TypeScript strict mode
- Implementare error handling per tutte le chiamate API
- Validare responses dal server usando Zod schemas
- Gestire stati di loading e error nell'UI



Backend Development

Stack Tecnologico

Server: Go 1.22 con Gorilla Mux router

- Redis client (go-redis/v9) per caching e sessioni
- MongoDB driver per persistenza dati
- Elastic APM per monitoring performance
- JSON Schema validation per API security

Dipendenze Principali:

- github.com/gorilla/mux: HTTP routing
- github.com/redis/go-redis/v9: Redis client
- go.mongodb.org/mongo-driver: MongoDB operations
- github.com/santhosh-tekuri/jsonschema/v5: Schema validation
- go.elastic.co/apm/v2: Performance monitoring

Middleware Architecture

Validator Middleware ([server/middleware/validator.go](#)):

- Route whitelisting: Solo endpoint con schema JSON sono accessibili
- Automatic validation: Body requests validati contro schemi in [comms/server/public/](#)
- Security by default: Rotte non schemate ritornano 401 Unauthorized

Implementazione esistente:

- Carica schemi JSON da directory [comms/server/public/](#)
- Valida body JSON contro schema corrispondente
- Ripristina request body per handler successivi



Security & Validation

Schema-Based API Security

Directory Structure ([comms/server/public/](#)):

- Ogni endpoint deve avere file `{endpoint}.req.json` per validation
- Schema definiscono esattamente struttura richieste accettate
- Validazione automatica tramite middleware Go

Route Whitelisting: Rotte attualmente esposte (definite in [middleware/validator.go](#)):

- [POST /auth/session](#): Creazione sessione
- [GET /auth/validate](#): Validazione token
- [GET /health](#): Health check

Frontend Schema Validation:

- Schemi in [comms/frontend/](#) per contratti frontend
- **Da implementare:** Validazione Zod per type safety

Redis Security

Accesso controllato tramite ACL:

- **Frontend:** Solo operazioni read (+get +exists +ttl +pttl +type +ping)
 - **Backend:** Full access per gestione sessioni e cache
 - **Admin UI:** Access completo per debugging
-

Monitoring & Analytics

Elastic APM Integration

Server-side: Automatic tracing tramite APM modules

- [go.elastic.co/apm/module/apmgorilla/v2](https://www.elastic.co/apm/module/apmgorilla/v2): Router instrumentation
- [go.elastic.co/apm/module/apmmongo/v2](https://www.elastic.co/apm/module/apmmongo/v2): Database query tracing
- Accessible via <http://apm.localhost>

Frontend APM: Configuration in [src/apm-rum-config.js](#)

- Real User Monitoring per performance tracking
- Error tracking e debugging
- User interaction analytics

Log Aggregation

Fluent Bit: Container log collection e parsing

- Automatic service identification tramite [container_name.lua](#)
- Custom parsers per MongoDB, NGINX, Go server logs
- Storage in Elasticsearch con service metadata

Kibana Dashboards: <http://kibana.localhost>

- Game performance metrics
 - Infrastructure health monitoring
 - Error tracking e debugging
-

Performance Optimization

Frontend Performance

Asset Management:

- Vite build optimization per production
- Asset bundling e tree-shaking automatico
- Static asset serving via NGINX

Game Performance:

- Phaser 3 performance best practices
- Sprite batching per reduced draw calls

- Memory management per long gaming sessions

Backend Performance

Redis Caching:

- Session storage per fast access
- Game state caching per reduced database load
- ACL-controlled access per security

Database Optimization:

- MongoDB connection pooling
- Efficient queries con proper indexing
- APM monitoring per query performance

Testing Strategy

Frontend Testing

Sandbox Environment:

- Testing isolato con scene personalizzate e stats controllate
- Debugging rapido senza impact su monitoring produzione
- Prototipazione di nuove mechanics prima dell'integrazione

Recommended Approach:

- Unit testing per game logic
- Integration testing per API communication
- E2E testing per critical user paths
- Performance testing per frame rate consistency

Tools da considerare:

- Jest per unit testing
- Cypress per E2E testing
- Lighthouse per performance auditing

Backend Testing

Testing Requirements:

- Unit tests per business logic
- Integration tests per database operations
- API testing con schema validation
- Load testing per performance

Considerazioni:

- Mock Redis e MongoDB per unit tests

- Test schema validation con esempi validi/invalidi
 - Performance benchmarking con Go testing tools
-

Deployment & CI/CD

Environment Management

Development: `docker-compose.dev.yml`

- Include sandbox environment per testing isolato
- Hot reload per development (frontend + sandbox)
- Full monitoring stack

Production: `docker-compose.prod.yml`

- Exclude sandbox per security e performance
- Optimized builds
- Production-grade configuration

NPM Registry Access:

- Credenziali configurate in `.env` per accesso a registry privati
- Email: `dev.progettiinnovativi.2025@gmail.com`
- Utilizzate per dependency management e private packages

Build Process

Frontend:

- TypeScript compilation
- Vite production build
- Asset optimization

Backend:

- Go binary compilation
- Docker image building
- Health check validation

Monitoring Setup

Health Checks:

- Application health endpoints
- Database connectivity verification
- Cache availability testing
- APM agent connectivity

Production Data Management

Pre-Production Cleanup: Prima del deployment produzione, eseguire pulizia sicura e intelligente:

```
# Backup automatico delle dashboard personalizzate
./scripts/kibana-dashboard-manager.sh export

# Pulizia selettiva preservando tutte le configurazioni
./scripts/cleanup.sh --production
```

Sistema di Pulizia Centralizzato: Il nuovo `cleanup.sh` offre modalità multiple con backup automatici:

- `--production`: Pulizia selettiva per deployment (preserva configurazioni)
- `--dev`: Pulizia completa per sviluppo
- `--soft`: Solo cache e build artifacts
- `--full`: Pulizia totale con conferma esplicita

Dati Rimossi (solo sviluppo/testing):

- `docker/volumes/mongodb/*` - Database giocatori di sviluppo
- `docker/volumes/logs/esdata/*` - Indici Elasticsearch di sviluppo
- `docker/volumes/logs/mongodb/*` - Log MongoDB di sviluppo
- `docker/volumes/fluent-bit-db/*.db*` - Cache log processing

Sempre Preservato:

- `docker/volumes/npm_data/` - Configurazioni proxy e domini
- `docker/volumes/npm_letsencrypt/` - Certificati SSL
- `docker/volumes/fluent-bit/` - Configurazione log aggregation
- `docker/volumes/kibana/config/` - Configurazione Kibana
- Dashboard Kibana (con backup automatico)
- `.env` files - Variabili ambiente

Workflow Deployment:

1. **Dashboard Backup:** `./scripts/kibana-dashboard-manager.sh export`
2. **Data Cleanup:** `./scripts/cleanup.sh --production`
3. **Config Verification:** `./scripts/check-persistent-config.sh`
4. **Production Deploy:** `./scripts/prod-rebuild.sh`

 **Documentazione Completa:** [Sistema di Pulizia Centralizzato](#)



Development Workflow

Feature Development Process

1. **Environment Setup:** `./scripts/dev-reinstall.sh`
2. **Prototipazione:** Sviluppa e testa in sandbox environment con scene customizzate
3. **Schema Definition:** Aggiorna schemi in `comms/` per nuovi endpoint
4. **Backend Implementation:** Implementa endpoint con validation
5. **Frontend Integration:** Aggiungi Zod validation per schemas
6. **Testing:** Verifica funzionalità su frontend reale (non solo sandbox)

7. **APM Monitoring:** Configura tracking per nuove features (solo frontend, non sandbox)
8. **Performance Validation:** Test response times e resource usage
9. **Production Deploy:** `./scripts/prod-rebuild.sh`

Code Quality Standards

Obbligatori:

- Schema validation per tutti i nuovi endpoint
- APM tracing per performance monitoring
- Structured logging per debugging
- Error handling completo
- Type safety (TypeScript + Go types)

Performance Requirements

Frontend:

- Consistent frame rate (target: 60fps)
- Fast asset loading (<2 seconds per level)
- Memory usage optimization

Backend:

- API response time <100ms (95th percentile)
- Database query optimization
- Efficient Redis usage

Communication Protocols

API Design:

- RESTful endpoints con schema validation
- Consistent error response format
- Proper HTTP status codes

Frontend-Redis:

- Direct connection con user `frontend` (read-only)
- Efficient caching strategies
- Connection pooling

Troubleshooting Workflow

Common Issues:

- Schema validation failures: Check `comms/` directory schemas
- Redis connection issues: Verify ACL configuration
- APM data missing: Check agent configuration
- Performance problems: Use Kibana dashboards

Debug Tools:

- Docker logs per service-specific debugging
 - Kibana per log aggregation analysis
 - APM dashboard per performance analysis
 - Redis UI per cache inspection
-

Questo documento deve essere mantenuto aggiornato con ogni modifica significativa all'architettura. Tutti gli sviluppatori devono familiarizzare con questi pattern prima di contribuire al progetto.